

Informe Tecnico De Sesión - Clean4Jesus

Fecha: 14 de julio de 2026 **Version del proyecto:** 1.3.3 (`versionCode 38`) **Foco de la sesión:** Login, Comunidad, Supabase, seguridad y validación profesional **Estado final:** Arquitectura backend y Auth endurecida; APK detenida por hallazgos de red team pendientes.

1. Resumen Ejecutivo

Durante esta sesión Clean4Jesus paso de tener una base inicial de login y comunidad a contar con una arquitectura real de autenticación, base de datos, permisos y moderación sobre Supabase.

La parte más importante no fue solamente “hacer que el login abra”. Se trabajó para que un usuario no pueda ver o modificar información de otra persona, para que una sesión cerrada deje de servir, para que eliminar una cuenta requiera comprobar de nuevo la contraseña y para que las acciones de moderación queden registradas y no puedan falsificarse fácilmente.

Después se ejecutó una batería amplia de pruebas. Las pruebas automáticas y los ataques contra Supabase pasaron. Sin embargo, dos revisores adversariales encontraron riesgos que todavía deben corregirse antes de generar la siguiente APK. Por esa razón no se entregó una APK al final: se prefirió detener el build antes que presentarte una versión sabiendo que contenía riesgos relevantes.

En lenguaje sencillo: **el edificio ya tiene estructura, puertas y cerraduras reales; ahora la inspección encontró algunas ventanas que todavía deben reforzarse antes de invitar usuarios.**

2. Que Es Supabase Y Por Que Se Agrego

Hasta ahora varias funciones de Clean4Jesus vivían solamente dentro del teléfono. Eso funciona para Refugio y Palabra, pero no alcanza para Comunidad, porque una comunidad necesita que varias personas puedan compartir información entre dispositivos.

Supabase cumple tres trabajos principales:

1. **Autenticación:** sabe quien inicio sesión y comprueba su identidad.
2. **Base de datos:** almacena perfiles, publicaciones, comentarios, apoyos en oración y reportes.
3. **Seguridad:** decide que información puede leer o modificar cada persona.

La analogía más sencilla es esta:

- La aplicación es la recepción del edificio.
- Supabase Auth es el documento de identidad y la tarjeta de acceso.

- Postgres es el archivo donde se guarda la informacion.
- RLS son las cerraduras individuales de cada habitacion.
- Las Edge Functions son empleados autorizados para tareas delicadas que la recepcion no puede ejecutar por si sola.

3. Sistema De Login Construido

3.1 Registro e inicio de sesion

Se preparo el flujo para que una persona pueda:

- crear una cuenta con correo y contrasena;
- confirmar su correo;
- iniciar sesion;
- mantener una sesion de forma segura;
- cerrar sesion;
- recuperar su contrasena;
- eliminar su cuenta con una comprobacion adicional.

3.2 Sesion guardada de forma segura

Los datos de sesion se almacenan con `SecureStore`, el almacenamiento protegido del sistema operativo. No se dejan como texto normal dentro de `AsyncStorage`.

Esto importa porque una sesion contiene credenciales temporales. Si alguien obtuviera una copia ordinaria del almacenamiento de la app, no deberia encontrar esas credenciales expuestas facilmente.

3.3 PKCE y deep links

Se configuro PKCE para los flujos de autentificacion. PKCE evita que otra aplicacion robe y reutilice facilmente el codigo temporal que llega durante una confirmacion o recuperacion.

El enlace de retorno usa:

```
clean4jesus://auth/callback
```

Eso significa que, despues de confirmar el correo o recuperar la contrasena, Android puede devolver al usuario directamente a Clean4Jesus.

3.4 Recuperacion de contrasena endurecida

La pantalla para crear una contrasena nueva solo queda autorizada cuando Supabase emite el evento real de recuperacion. No basta con abrir manualmente una ruta de la app.

La autorizacion:

- queda ligada al usuario correcto;

- dura un máximo de 15 minutos;
- se conserva si ocurre un error recuperable;
- se elimina al terminar correctamente.

3.5 Eliminación de cuenta protegida

Eliminar una cuenta no depende únicamente de que el usuario haya iniciado sesión alguna vez. La Edge Function vuelve a comprobar:

- la identidad del usuario;
- que la sesión siga activa;
- la contraseña ingresada en ese momento;
- que el ID solicitado corresponda al usuario autenticado.

La clave administrativa de Supabase nunca se incluye dentro del APK.

4. Comunidad Real Construida

La arquitectura de Comunidad contempla:

- perfiles públicos limitados;
- publicaciones y testimonios;
- pedidos de oración;
- reacciones de apoyo;
- comentarios;
- reportes;
- estados de contenido visible u oculto;
- roles privados de moderador y administrador;
- registro de acciones administrativas.

Se mantuvo fuera de Supabase la información más sensible del producto:

- historial de navegación o bloqueo;
- actividad detectada por Refugio;
- racha limpia privada;
- progreso local de Palabra mientras no exista una decisión de sincronización.

Crear una cuenta no significa subir automáticamente estos datos.

5. Seguridad De La Base De Datos

5.1 RLS

RLS significa `Row Level Security` o seguridad por fila. Es una regla aplicada directamente por la base de datos.

Ejemplo: aunque alguien altere el APK e intente pedir la publicación privada de otra persona, Postgres vuelve a comprobar si tiene permiso. La seguridad no depende solamente de ocultar un botón en la pantalla.

5.2 Mínimo privilegio

Cada usuario recibe solamente los permisos necesarios. Un usuario normal no puede convertirse en moderador modificando una solicitud, ni alterar campos controlados por el sistema.

También se agregaron permisos por columna. Esto evita que una aplicación modificada intente cambiar campos como estado, rol o datos sensibles.

5.3 Sesiones realmente revocadas

Un token puede parecer válido matemáticamente hasta su fecha de expiración. Por eso las operaciones sensibles no comprueban solo el token: también revisan que su `session_id` siga existiendo en `auth.sessions`.

Resultado: si una sesión fue cerrada o revocada, ese token antiguo deja de servir aunque todavía no haya expirado.

5.4 Moderación segura

La moderación se construyó con:

- roles privados;
- versión esperada para evitar que dos moderadores pisen cambios;
- claves de idempotencia para no ejecutar dos veces la misma acción;
- evidencia durable;
- auditoría inmutable;
- MFA `aa12` para restauraciones administrativas.

En términos simples: las decisiones importantes dejan un recibo, tienen control de concurrencia y no confían en el nombre de usuario enviado desde el teléfono.

5.5 Migraciones desplegadas

Se sincronizaron ocho migraciones entre el proyecto y Supabase. Una migración es un cambio versionado de la estructura o las reglas de la base de datos.

Esto permite saber exactamente como evoluciona la base y repetir la configuracion sin depender de cambios manuales olvidados.

6. Edge Functions Desplegadas

Se desplegaron dos funciones de servidor:

1. `delete-account` : elimina una cuenta despues de reautenticar al usuario.
2. `moderate-community` : ejecuta acciones de moderacion con permisos y auditoria.

Estas tareas viven en el servidor porque necesitan privilegios que nunca deben viajar dentro de la app movil.

7. Pruebas Ejecutadas

7.1 TypeScript

Resultado: **aprobado**.

TypeScript revisa que el codigo use correctamente los tipos de datos. Ayuda a detectar, antes de abrir la app, errores como tratar un valor vacio como si siempre existiera.

7.2 Pruebas unitarias

Resultado: **58 de 58 aprobadas**.

Estas pruebas validan piezas pequenas y aisladas: reglas de sesion, recuperacion, persistencia, servicios y contratos internos.

7.3 Pruebas E2E

Resultado: **7 de 7 aprobadas**.

E2E significa `end to end` : se abre la aplicacion como lo haria un usuario y se revisan recorridos completos. Tambien se comprobaron los checkpoints visuales del footer, Planes y Comunidad en anchos Android comunes.

7.4 Exportacion web

Resultado: **aprobado**.

Metro proceso 1.302 modulos y genero correctamente el bundle web. Esto confirma que no existen importaciones rotas ni errores de empaquetado JavaScript en esa plataforma.

7.5 Lint de Supabase

Resultado: **sin errores del esquema**.

El linter reviso los esquemas `extensions` , `private` y `public` del proyecto remoto.

7.6 Ataques remotos de seguridad

Se ejecutaron dos baterias contra el Supabase real usando identidades temporales.

Bateria de seguridad, 8 etapas:

- creacion de identidades aisladas;
- comprobacion de RLS;
- creacion de reportes;
- moderacion atomica;
- idempotencia;
- concurrencia;
- separacion de roles;
- eliminacion protegida.

Bateria negativa, 8 etapas:

- rechazo de edicion sobre contenido oculto;
- rechazo de comentarios bajo publicaciones ocultas;
- rechazo de versiones faltantes;
- rechazo de reutilizacion de idempotencia con otro contenido;
- conservacion de evidencia;
- limites bajo concurrencia;
- invalidacion de una sesion revocada;
- invalidacion del token despues de eliminar una cuenta.

Resultado total remoto: **16 etapas aprobadas.**

Las cuentas temporales fueron eliminadas al terminar.

8. Revision Adversarial

Antes de generar el APK se desplegaron dos revisores separados:

- uno especializado en Android, VPN, Accesibilidad y empaquetado;
- otro especializado en UX, Auth, privacidad, Comunidad y regresiones.

Su trabajo no fue demostrar que la app funcionaba, sino intentar romperla.

8.1 Hallazgos del Refugio

Se encontraron riesgos preexistentes dentro del checkpoint congelado:

1. La app puede guardar el Refugio como activo antes de confirmar que Android concedio realmente el permiso VPN.
2. El estado VPN consulta una bandera persistida y puede producir un falso positivo despues de ciertos reinicios o cierres abruptos.
3. Algunas excepciones de YouTube o banca dependen de texto visible y podrian permitir una evasion si una pagina maliciosa incluye esas palabras.
4. El texto de privacidad dice que la app "no lee mensajes", pero Accesibilidad necesita inspeccionar texto visible en memoria para detectar senales. No se demostro que ese contenido se suba, pero el copy debe ser mas preciso.
5. La VPN DNS actual solo usa un upstream efectivo y no implementa toda la redundancia declarada.

Estos hallazgos no se corrigieron porque Refugio es un checkpoint congelado y requiere autorizacion explicita antes de modificarlo.

8.2 Hallazgos de Comunidad

1. La consulta de reacciones permite obtener identificadores de quienes oraron por una publicacion. La UI solo necesita el conteo y la reaccion propia.
2. Faltan reportes directos para comentarios y categorias como autolesion, spam u otros riesgos.
3. Una publicacion propia muestra una accion de reporte que el servidor rechaza, pero todavia no ofrece editar o eliminar.
4. El copy de eliminacion usa la palabra "anonimizada", aunque la evidencia de moderacion se conserva pseudonimizada.
5. Los errores de sesion degradada no siempre se muestran claramente en Comunidad.

Estos puntos deben corregirse antes de abrir Comunidad a usuarios externos.

9. Por Que No Se Genero APK

El protocolo del proyecto dice que una APK no debe utilizarse para descubrir errores obvios. Primero se ejecutan pruebas, despues red team y finalmente build.

La bateria automatica aprobo, pero el red team emitio un resultado **NO-GO**. Generar la APK ignorando ese resultado habria contradicho el proceso profesional solicitado.

No significa que todo el trabajo haya fallado. Significa que el control de calidad funciono: encontro riesgos antes de entregarte una version para probar.

10. Lo Que No Se Toco

Durante el trabajo de Auth y Supabase se conservaron los checkpoints aprobados:

- footer Android 1.2.24;
- catalogo y detalle de Planes;

- visual de Palabra;
- logica funcional del Refugio;
- pantalla de interrupcion;
- allowlists de bancos y YouTube;
- proyecto externo `Learning path TAAG Project`.

No se ejecuto `npm audit fix --force`, porque actualmente propone saltar Expo SDK 54 a SDK 57. Esa actualizacion requiere una fase separada y pruebas de Android nativo.

11. Estado Real Al Cerrar

Aprobado

- arquitectura Auth;
- almacenamiento seguro de sesion;
- PKCE y deep links;
- recuperacion endurecida;
- eliminacion con reautenticacion;
- RLS y permisos por columna;
- sesiones revocables;
- moderacion concurrente;
- Edge Functions;
- 58 pruebas unitarias;
- 7 pruebas E2E;
- 16 etapas de ataques remotos;
- exportacion y lint de base.

Pendiente antes de APK beta

- autorizar y corregir los hallazgos del checkpoint Refugio;
- ocultar identidades de reaccion en Comunidad;
- completar reportes y salida para contenido propio;
- corregir el copy de retencion/anominizacion;
- integrar SMTP real;
- integrar CAPTCHA;
- construir consola interna de moderacion;
- probar confirmacion y recuperacion mediante correo real en Android.

12. Proximo Orden Recomendado

1. Abrir de forma controlada el checkpoint de Refugio y corregir solamente los hallazgos aprobados por producto.
2. Crear una migracion aditiva que entregue conteos de oracion sin exponer identidades.
3. Completar reportes de comentarios, autolesion y spam.
4. Corregir copy y manejo de errores de Auth.
5. Repetir las 58 unitarias, 7 E2E y 16 etapas remotas.
6. Ejecutar de nuevo dos revisores adversariales.
7. Generar APK local de prueba y entregar el comando `npm run dev-client`.
8. Configurar SMTP y CAPTCHA antes de una beta externa.

13. Glosario Para No Programadores

- **APK:** archivo instalable de Android.
 - **Auth:** sistema que identifica usuarios y administra sesiones.
 - **Backend:** parte que vive en servidores y no dentro del telefono.
 - **Base de datos:** lugar estructurado donde se guarda informacion.
 - **Deep link:** enlace que abre una pantalla especifica de la app.
 - **Edge Function:** funcion segura que se ejecuta en el servidor.
 - **E2E:** prueba que recorre la app como un usuario.
 - **Idempotencia:** proteccion para que repetir una solicitud no duplique una accion.
 - **JWT:** credencial temporal que demuestra una sesion.
 - **MFA:** segundo factor de autenticacion.
 - **Migracion:** cambio versionado de la base de datos.
 - **PKCE:** proteccion para flujos de login y recuperacion.
 - **RLS:** reglas de la base que controlan cada fila segun el usuario.
 - **SMTP:** servicio encargado de enviar correos reales.
 - **Supabase:** plataforma usada para Auth, Postgres y funciones de servidor.
-

Conclusion: La sesion dejo una base de Auth y Comunidad considerablemente mas profesional y comprobada. La decision correcta de cierre es conservar [1.3.3](#), no generar una APK todavia y resolver los hallazgos adversariales antes de la siguiente candidata instalable.